CATALYST: CATalogue-driven LLM sYstem for Spatial Tiling*

Tarlan Bahadori
University of California, Riverside
Riverside, California
tbaha001@ucr.edu

Ahmed Eldawy
University of California, Riverside
Riverside, California
eldawy@ucr.edu

ABSTRACT

Geospatial data constitutes a large and growing fraction of publicly available open data, yet effectively using it remains challenging due to difficulties in dataset discovery, scalable visualization, and map styling. We demonstrate CATALYST, a catalogue-driven and LLM-assisted system for interactive exploration of large-scale geospatial repositories. Given a high-level natural-language prompt, CATALYST retrieves the most relevant datasets through semantic search, visualizes them efficiently using a visualization index, and automatically generates cartographically meaningful styles aligned with user intent. The proposed demo scenario gives the audience a chance to test enriched semantic data retrieval and styling with simple prompts and encourages more research in the area of geospatial data analysis.

PVLDB Reference Format:
Tarlan Bahadori and Ahmed Eldawy. CATALYST: CATalogue-driven LLM sYstem for Spatial Tiling. PVLDB, 19(12): , 2026.
doi:

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/tarlaun/starlet.

1 INTRODUCTION

Open data initiatives and advances in data collection technologies have led to an unprecedented growth in publicly available data, with geospatial data comprising a significant portion of them. For instance, Data.gov hosts nearly 300,000 geospatial datasets, accounting for more than 73% of all datasets on the platform. These datasets are used in a wide range of applications, including environmental monitoring, transportation systems, precision agriculture, urban planning, and political, playing a critical role in both scientific research and real-world decision-making.

Despite the wide availability and usefulness of geospatial datasets, there are three challenges that limit users from fully utilizing them. First, it is difficult to identify and retrieve datasets relevant to a specific application due to incomplete, inconsistent, or missing metadata. Second, to assist in using these datasets, users often need to visualize and explore them on a map, but their large size, reaching tens to hundreds of gigabytes, results in unscalable visualization, negatively impacting productivity. Third, to deliver the message effectively, users need to carefully choose suitable map styling options, but the richness of the datasets and the lack of metadata make this step challenging. Collectively, these challenges create significant barriers to the full use of geospatial data in both research and practical applications.

This paper demonstrates CATALYST, a catalogue-driven and LLM-assisted system that overcomes the above challenges and enables users to effectively utilize large-scale geospatial datasets. Rather than requiring users to choose a dataset explicitly, CATALYST supports high-level, intent-driven queries such as *"show areas with high wildfire impact"* or *"visualize theft-related crime hotspots"*. Given such prompts, CATALYST first performs semantic retrieval over an enriched catalogue to identify the most relevant datasets and attributes. It then leverages a specialized tile-based index to load and render the selected dataset on an interactive map, enabling scalable, low-latency visualization even for datasets at the scale of tens to hundreds of gigabytes. Finally, it automatically generates cartographically meaningful styling that align with the user's intent, completing an end-to-end pipeline from high-level query to interactive visualization.

Figure 1 gives a high-level overview of CATALYST which runs in two phases, the *preprocessing* phase and the *online querying* phase. In the *preprocessing phase*, CATALYST connects to a geospatial data lakehouse to perform two operations. First, it automatically enriches datasets with descriptive metadata derived from dataset statistics and summaries, e.g., attribute names and types, top-k values, and histograms. This enriched metadata is then converted into AI-friendly embeddings and stored in a vector DBMS to enable semantic retrieval at query time. Second, it builds a tile-based visualization index by constructing tile histograms and a multi-resolution pyramid, allowing the system to precompute and materialize dense or large tiles while generating smaller tiles on the fly, thus balancing precomputation overhead and interactive visualization latency. To ensure bounded tile sizes, sampling techniques are applied during tile generation. Unlike other methods [4], CATALYST generates vector tiles that allow custom map styles in the query phase. In the *online query phase*, a user provides a free-text prompt, which is used to retrieve the most relevant datasets via the vector DBMS. The system then loads the corresponding visualization index and uses an LLM to generate appropriate styling specifications for the data. The querying phase is conversation-driven, allowing users to iteratively refine both the dataset selection and the visualization style until they obtain results that best match their analysis needs.

2 SYSTEM DESIGN

As illustrated in the system pipeline (Figure 1), our framework consists of an offline preprocessing phase and an online query phase. During preprocessing, we construct two complementary

*This work is supported in part by the National Science Foundation (NSF) under grant IIS-2046236
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 12 ISSN 2150-8097.
doi:

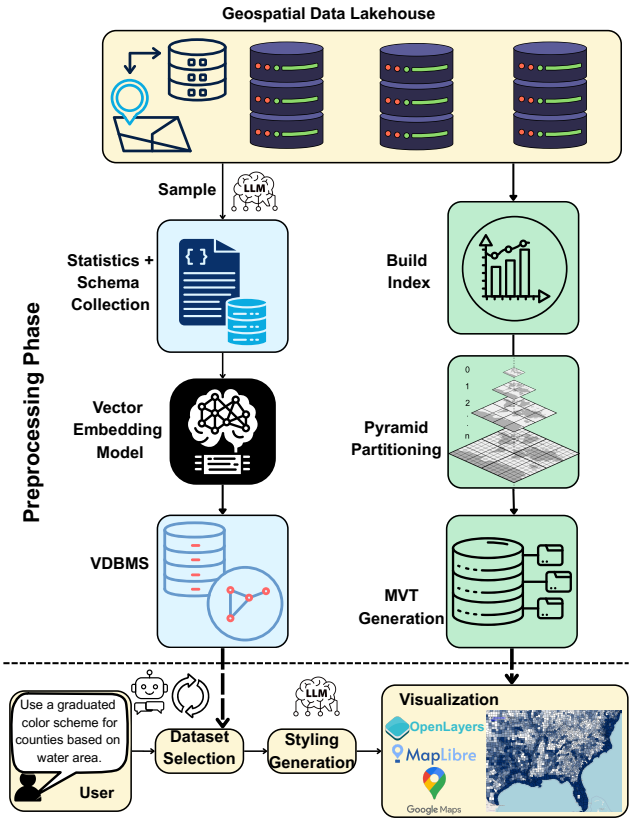


Figure 1: Overview of the catalogue-driven visualization pipeline. The system builds tile histograms and a zoom pyramid index to support hybrid tile generation (precompute dense tiles, generate small tiles on demand) while also maintaining an enhanced-schema embedding index for scalable dataset discovery and LLM-assisted styling.

indices in parallel—(1) an enhanced-schema semantic index and (2) a tile histogram index—alongside multi-resolution MVT generation. The online phase leverages these indexes to support efficient dataset discovery, styling, and visualization.

2.1 Semantic Index

The first component is a semantic index over datasets, designed to support scalable discovery and attribute selection. For each dataset, we compute an *enhanced schema* that captures attribute names, inferred types, and compact statistical summaries (e.g., distributions, top-k values, and numeric ranges).

We then construct a textual descriptor by combining dataset metadata with these summaries. This descriptor is embedded using a vector embedding model and stored in a PostgreSQL vector database (VDBMS). This index acts as a semantic index over the dataset repository, enabling similarity-based retrieval.

At query time, a user request is embedded and matched against this index to retrieve the top-k most relevant datasets. These candidates are passed to the LLM, which selects the most appropriate dataset and attribute(s) for visualization. This *search-then-reason* design avoids sending the entire catalogue to the LLM, reducing both latency and token usage.

2.2 Visualization Index

To support scalable interactive visualization, we adopt the standard pyramid tile structure used in modern web-based maps, where the spatial domain is recursively partitioned into tiles across multiple zoom levels (typically 20 levels). While fully materializing this pyramid enables low-latency access, it incurs prohibitive storage overhead due to the exponential growth in the number of tiles.

To address this, we construct a *tile histogram index* over the pyramid to capture data distribution across tiles. For each tile (z, x, y), we estimate (i) the number of intersecting features and (ii) the expected tile size in bytes. This is computed by mapping features to intersecting tiles and aggregating statistics such as vertex counts, resulting in a sparse summary of spatial density across zoom levels.

Using this index, we classify tiles into *dense* and *sparse* categories. Dense tiles are those likely to exceed size thresholds, while sparse tiles are small and inexpensive to generate on demand. This classification enables a hybrid materialization strategy inspired by prior work on pre-rendered tiles [4], which we extend to support vector tiles (MVT).

In parallel with index construction, we perform pyramid partitioning and tile generation. For each dense tile, we extract the corresponding subset of features and apply bounded-size reservoir sampling to ensure that tile sizes remain within practical limits. Dense tiles are materialized offline as MVT files and stored in an on-disk index for efficient retrieval.

To support on-demand generation of sparse tiles, we additionally build a spatial index over all features. At query time, this index allows the system to efficiently retrieve features intersecting a requested tile and generate the corresponding MVT dynamically. Together, these components enable low-latency visualization while balancing storage overhead and computation cost.

Online execution. At query time, the user provides a free-text prompt and the system should retrieve the intended dataset and visualize it. This is done through the following steps:

(1) We embed the user-provided prompt and use the semantic index to retrieve candidate datasets and guide LLM-based dataset and attribute selection.
(2) We feed the user prompt along with the enriched metadata to an LLM to refine the selection to the most relevant dataset.
(3) The corresponding dataset is loaded in the web browser with the help of the visualization index.
(4) We use LLM to specify a rendering style for the dataset and display it to the user.

The user can refine the process above by providing additional prompts with a conversation-based interface until the desired styling and visualization of the data are produces. This design allows the system to scale to large repositories while maintaining interactive performance.

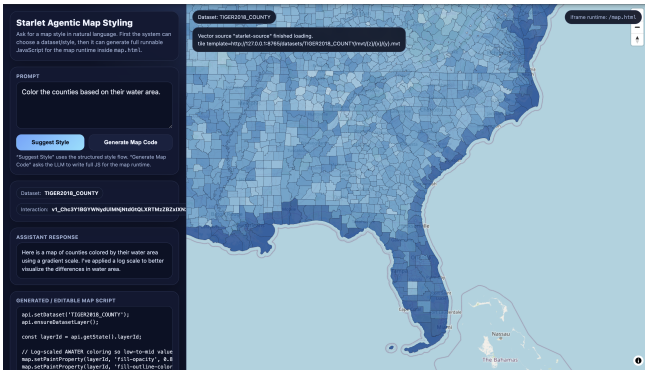


Figure 2: Example of using the LLM for selecting the most suitable dataset for user query: "Color the counties based on their water area." and generating styling code for the map. Assistant response: "Here is a map of counties colored by their water area using a gradient scale. I've applied a log scale to better visualize the differences in water area."

3 DEMONSTRATION SCENARIOS

We demonstrate Catalyst through several interactive scenarios that highlight the system's ability to discover datasets, generate visualizations, and support conversational refinement. The demonstration integrates the Google Gemini language model, although the architecture is designed to support alternative models.

3.1 Scenario 1: Upload and Process Dataset

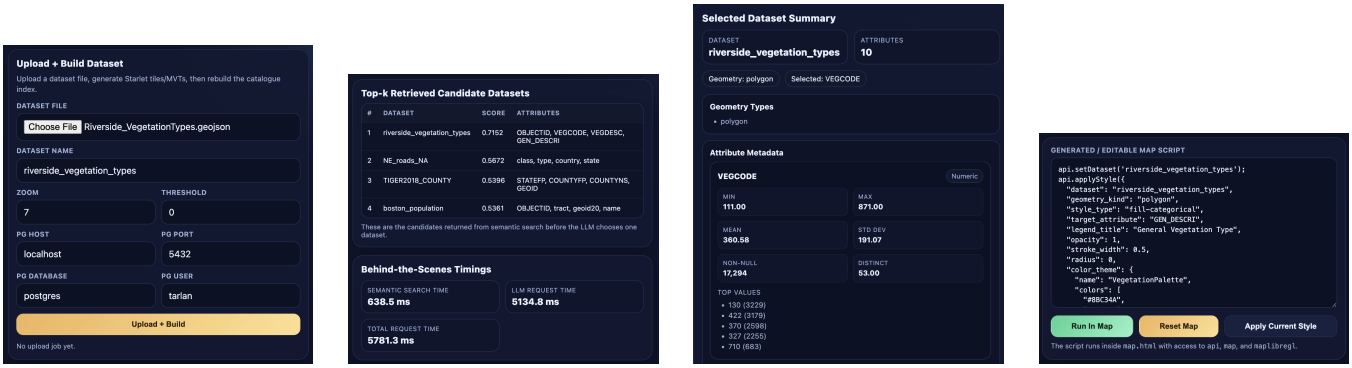
In the first scenario, the user introduces a new dataset into the system through the web interface. Instead of relying on pre-existing data, the user uploads a raw geospatial file (e.g., GeoJSON or GeoParquet) and provides basic configuration parameters such as maximum zoom level and tile size threshold of MVT generation (figure 3a). Upon upload, the system automatically triggers a preprocessing pipeline. First, the dataset is partitioned into spatial tiles using the Starlet tiling engine, which organizes the data into manageable spatial chunks. In parallel, vector tiles (MVTs) are generated to support efficient visualization across zoom levels. Once tiling is complete, the system constructs an enriched dataset representation by computing schema metadata and statistical summaries for each attribute. This information is then embedded into a vector space and added to the catalogue index. Optionally, the embeddings are synchronized with a PostgreSQL vector database (e.g., pgvector) to enable scalable semantic retrieval. Users can see the progress and will be notified when the dataset is ready for visualization. This scenario highlights the system's ability to seamlessly integrate new datasets into its ecosystem. By automating tiling, indexing, and metadata generation, the system transforms raw uploaded data into a fully queryable and visualizable resource without requiring manual preprocessing.

3.2 Scenario 2: Iterative Visualization Refinement

The user begins by entering a prompt into the system, for example "Color Riverside based on vegetation types". The styled visualization for this prompt can be seen in figure 4 The system will match this query against the semantic index of vector embeddings first to find the most relevant dataset, e.g., Riverside_VegetationTypes, and then sends the metadata of that dataset alongside the user NL query to the LLM styling agent. Figure 3b shows top retrieved candidate datasets for the user query. Additionally, as shown in figure 3c, users can view the metadata of the selected attribute to better understand styling options. There are two styling modes available, first one will generate a structured style, in this mode, the LLM will return a structured JSON file that is translated to JavaScript styling code in the backend. In the second mode, the LLM will directly send a JavaScript snippet that will run in the map runtime. This mode is explored in scenario 3. After the initial visualization is generated, in the structured mode, the user can refine the map using conversational instructions. For example, the user might request "Only show the top 5 most prominent vegetation types" or "Change the color theme to red." The session manager preserves the context of the previous visualization, allowing the system to interpret these instructions without requiring the user to restate the dataset or attribute. The language model updates the styling specification accordingly, and the map renderer immediately reflects the changes. This scenario demonstrates how conversational memory enables iterative exploration and visualization refinement.

3.3 Scenario 3: Full Code Generation for Visualization

In the third scenario, the user interacts with the system through natural language to directly generate executable visualization code, rather than receiving only a structured style suggestion. In this mode, the LLM can provide a basis for users who may want to modify the JavaScript code further. For example, the user may send a prompt such as "Color counties based on water area" (figure 2) or "Highlight neighborhoods in Boston based on population density." As in the previous scenario, the system first identifies the relevant dataset and attributes using semantic retrieval over the catalogue index. However, instead of returning a declarative style specification, the system invokes the code generation module to produce a complete JavaScript snippet that can be executed within the map runtime. An example for this can be seen in figure 4. The generated code includes dataset selection, styling logic, and rendering instructions (e.g., MapLibre layer configuration), and is immediately executed in the client. This allows the user to see the resulting visualization without manually translating high-level intent into implementation details. Importantly, the system leverages both the structured style representation and the dataset metadata during code generation. This ensures that the produced code is consistent with the underlying data schema and compatible with the rendering pipeline. In cases where naive linear mappings would produce poor visual differentiation (e.g., highly skewed attributes), the system can incorporate transformations such as logarithmic scaling or custom interpolation. Interactive multi-turn conversation with the LLM agent allows users to quickly make such modifications even if they are unfamiliar with the styling code. This scenario demonstrates the system's ability to move beyond recommendation and into full automation, effectively acting as a co-pilot that translates user intent into executable visualization logic.



(a) (b) (c) (d)

Figure 3: a Upload configuration interface. User can set parameters such as maximum zoom level and tile size threshold for MVT generation. b Top-k dataset retrieval candidates from semantic index (vector database) matching user query. Additionally, time of semantic retrieval and LLM query. c Metadata summary of the selected dataset. Statistical information of all attributes can be viewed here. d Automatically generated full map styling code. Users can manually modify this code or continue interactive conversation with the system until they get their desired styling.

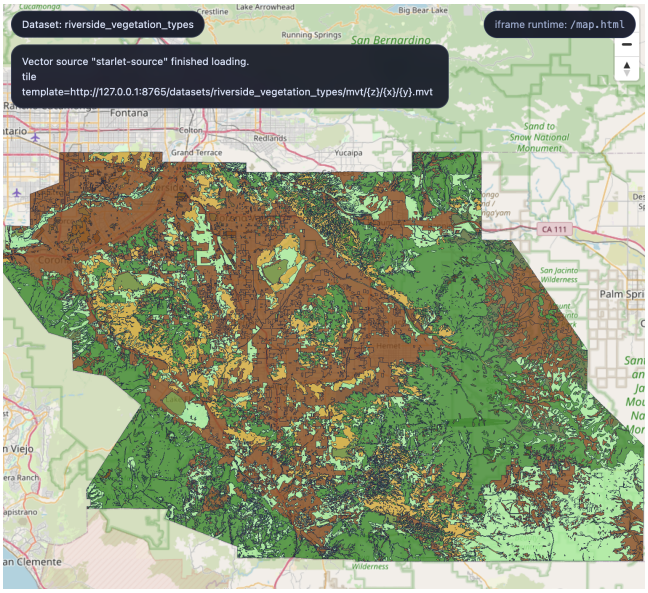


Figure 4: Visualization of LLM-generated styling code, showing vegetation types in Riverside, for user prompt: "Color Riverside based on vegetation types". The system has identified most suitable dataset and attribute for this query and prompted LLM agent to generate styling code accordingly.

4 FUTURE WORK

An important direction for future work is enabling *cross-dataset exploration*, where the system can seamlessly integrate and visualize information from multiple datasets within a unified map. This would require extending the current catalogue and indexing framework to support dataset alignment, schema matching, and joint querying across heterogeneous sources. Another key avenue is the *safe execution of LLM-generated code*. While our system leverages LLMs to produce visualization logic, executing such code raises security and reliability concerns. Future work will explore sandboxing mechanisms, static and dynamic code analysis, and constrained execution environments to ensure that generated code can be safely validated and run without compromising system integrity. More broadly, we aim to investigate tighter integration between the LLM reasoning pipeline and the underlying indexing infrastructure, allowing for more robust, interpretable, and controllable generation of visualization workflows.

REFERENCES

[1] 2025. Interactive Maps with OpenLayers. https://openlayers.org/. OpenLayers Documentation.

[2] Tarlan Bahadori, Sai Sreekar Sarvepalli, and Ahmed Eldawy. 2025. LASEK: LLM-Assisted Style Exploration Kit for Geospatial Data. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5435–5438. https://doi.org/10.14778/3750601.3750690

[3] Ahmed Eldawy et al. 2021. Beast: Scalable Exploratory Analytics on Spatio-temporal Data. In *ACM International Conference on Information and Knowledge Management (CIKM '21)*. 3796–3807.

[4] Saheli Ghosh and Ahmed Eldawy. 2022. AID*: A Spatial Index for Visual Exploration of Geo-Spatial Data. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2022), 3569–3582. https://doi.org/10.1109/TKDE.2020.3026657

[5] Saheli Ghosh, Tin Vu, Mehrad Amin Eskandari, and Ahmed Eldawy. 2019. UCR-STAR: the UCR spatio-temporal active repository. *SIGSPATIAL Special* 11, 2 (Dec. 2019), 34–40. https://doi.org/10.1145/3377000.3377005

[6] Kevin Hu, Michiel A. Bakker, Stephen Li, Tim Kraska, and César Hidalgo. 2019. VizML: A Machine Learning Approach to Visualization Recommendation. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) *(CHI '19)*. 1–12. https://doi.org/10.1145/3290605.3300358

[7] Jinfeng Liu, Ziming Liu, Tanja Blascheck, Wencan Fan, and Nan Cao. 2023. Large language models for information visualization: Survey and opportunities. *Visual Informatics* 7, 4 (2023), 113–124.

[8] MapLibre Community. 2026. MapLibre GL JS (and Native). https://maplibre.org/. Accessed: 2026-03-26.

[9] Kanit Wongsuphasawat, Dominik Moritz, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. 2016. Towards a general-purpose query language for visualization recommendation. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics (HILDA'16@SIGMOD)*. Association for Computing Machinery, New York, NY, USA, Article 4, 6 pages.

[10] Yupeng Xie, Yuyu Luo, Guoliang Li, and Nan Tang. 2024. HAIChart: Human and AI Paired Visualization System. *Proc. VLDB Endow.* 17, 11 (July 2024), 3178–3191.

[11] Jia Yu and Mohamed Sarwat. 2021. GeoSparkViz: a cluster computing system for visualizing massive-scale geospatial data. *The VLDB Journal* 30, 2 (Jan. 2021), 22.